

# Deep Learning Meets Queue-Reactive

A Framework for Realistic Limit Order Book Simulation [1]

Hamza Bodor and Laurent Carlier (2025)

arXiv:2501.08822

Paul Archer

CID: 06054057

Thibault Marty

CID: 06055275

Rebecca Laïk

CID: 01776263

## Contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Summary of the article</b>                                       | <b>1</b>  |
| 1.1      | Limit Order Books and Baseline Queue-Based Models . . . . .         | 1         |
| 1.2      | A Deep Learning Extension of the Queue-Reactive Framework . . . . . | 1         |
| 1.3      | Multidimensional Deep Queue-Reactive Model . . . . .                | 2         |
| <b>2</b> | <b>Numerical experiments</b>                                        | <b>3</b>  |
| 2.1      | Data and preprocessing . . . . .                                    | 3         |
| 2.2      | Queue-Reactive (QR) Model . . . . .                                 | 3         |
| 2.3      | Deep Queue-Reactive (DQR) Model . . . . .                           | 4         |
| 2.4      | Multidimensional Deep Queue-Reactive (MDQR) Model . . . . .         | 5         |
| <b>3</b> | <b>Can MDQR learn rough volatility patterns ?</b>                   | <b>7</b>  |
| 3.1      | Realized volatility estimation . . . . .                            | 8         |
| 3.2      | Hurst exponent estimation . . . . .                                 | 8         |
| 3.3      | Observations . . . . .                                              | 9         |
| <b>4</b> | <b>Critical Discussion</b>                                          | <b>10</b> |
|          | <b>Appendix</b>                                                     | <b>12</b> |
| A        | INTC : Summary Statistics . . . . .                                 | 12        |
| B        | Normalised Order Size Distribution . . . . .                        | 12        |
| C        | CNN Variant of MDQR . . . . .                                       | 12        |
| D        | $\hat{\eta}_t$ estimation . . . . .                                 | 14        |
| E        | Intraday volatility clustering . . . . .                            | 15        |

# 1 Summary of the article

## 1.1 Limit Order Books and Baseline Queue-Based Models

Electronic limit order books (LOBs) are the primary mechanism for price formation in modern markets. Prices emerge from heterogeneous agents submitting various order types, making the modeling of LOB dynamics essential for understanding price formation and liquidity. A limit order book aggregates trading intentions at discrete price levels through three types of events: *limit orders*, *market orders*, and *cancellations*. At each price level, the outstanding volume forms a *queue*, and the book is organized into bid and ask sides. Modeling limit order books is challenging due to strong temporal dependencies and endogenous interactions between market participants, leading to features such as clustered order flow and non-linear dynamics.

A widely used baseline for limit order book modeling is the **Queue-Reactive (QR) model** introduced by Huang et al. (2015) [2]. The QR framework represents the order book as a collection of queues centered around the mid-price, with a fixed number  $K$  of price levels on each side. Each queue evolves independently through discrete events, whose arrival rates depend on the current queue size.

Formally, for each event type  $\eta \in \{L, C, M\}$ , the model specifies an intensity function  $\lambda^\eta(q)$ , governing the arrival of events conditional on the queue size  $q$ . Total activity is given by  $\Lambda(q) = \lambda^L(q) + \lambda^C(q) + \lambda^M(q)$ . This captures local liquidity dependence and yields a tractable likelihood under conditional Poisson assumptions of an event sequence  $\mathcal{E} = \{e_k\}_{k=1}^N$  :

$$\mathcal{L}(\{\lambda^\eta\} \mid \mathcal{E}) = \prod_{k=1}^N \exp(-\Lambda(q_k)\Delta t_k) \lambda^{\eta_k}(q_k), \quad (1)$$

The QR model is inherently local: each price level is independent and intensities depend only on its queue size. As a result, cross-level interactions and global order book dynamics cannot be captured. An extension, the Size-Aware Queue-Reactive (SAQR) model, incorporates order sizes but still relies on local queue information. These limitations motivate the introduction of richer state representations in subsequent models.

## 1.2 A Deep Learning Extension of the Queue-Reactive Framework

A natural generalization of the QR framework is the **Deep Queue-Reactive (DQR)** model introduced by Bodor and Carlier (2025)[1], which relaxes the reliance on local queue sizes by conditioning intensities on a richer state vector  $x_k$  rather than on  $q_k$  alone. This state vector may include multi-level queues, imbalance, spread, or past events. In the DQR model, the intensity functions are parameterized by neural networks, denoted  $\lambda_\theta^\eta(x_k)$ , where  $\theta$  represents the network parameters. The total intensity retains the same structure,  $\Lambda_\theta(x_k) = \sum_{\eta \in \{L, C, M\}} \lambda_\theta^\eta(x_k)$ . The likelihood formulation remains unchanged in structure, with the queue size  $q_k$  replaced by the state vector  $x_k$ . The negative log-likelihood minimized during calibration is given by

$$\text{loss}(\theta \mid \mathcal{E}) = \sum_{k=1}^N \left( \Lambda_\theta(x_k)\Delta t_k - \log \lambda_\theta^{\eta_k}(x_k) \right), \quad (2)$$

The DQR model is calibrated on high-frequency limit order book data from Euro-Bund Futures. To implement the DQR framework, the intensity functions are parameterized using a neural network that maps the state vector  $x_k$  to the corresponding order arrival intensities. The architecture relies on a shallow multi-layer

perceptron, balancing computational efficiency and expressiveness.

Table 1: Neural Architecture of the DQR Model

| Component         | Specification                                                         |
|-------------------|-----------------------------------------------------------------------|
| Input             | State vector $x_k$ (LOB features: queue sizes, imbalance, time, etc.) |
| Hidden Layer 1    | 128 units + Tanh activation                                           |
| Hidden Layer 2    | 32 units + Tanh activation                                            |
| Output Layer      | 3 units (intensities for L, C, M)                                     |
| Output Activation | ReLU                                                                  |
| Loss Function     | Negative log-likelihood (point process)                               |
| Optimizer         | Adam                                                                  |
| Learning Rate     | $10^{-5}$ to $10^{-3}$                                                |

Results show DQR captures key order flow dynamics. Including the previous event type enables excitation effects, while time-of-day features capture intraday seasonality. Performance is evaluated using log-likelihood, next-event accuracy, and time-to-event error. Ablation shows improvements with richer state representations, with best results obtained using both features. This motivates extending the model to a multidimensional framework with cross-level interactions and order sizes.

### 1.3 Multidimensional Deep Queue-Reactive Model

The **Multidimensional Deep Queue-Reactive (MDQR) model** introduced by Bodor and Carlier (2025)[1] extends DQR by modeling the full LOB as a multidimensional system. As in DQR, event arrival intensities are parameterized by neural networks and conditioned on a global state vector  $x_k$ . However, in MDQR this state is shared across all price levels, allowing cross-level interactions. An event at time  $t_k$  is represented as  $e_k = (\eta_k, \ell_k, \Delta t_k, s_k, x_k)$  where  $\eta_k$  denotes the event type,  $\ell_k$  the price level,  $\Delta t_k$  the inter-event time,  $s_k$  the order size, and  $x_k$  the global market state immediately before the event. The likelihood of the observed event sequence factorizes into two components,

$$\mathcal{L}(\theta \mid \mathcal{E}) = \prod_{k=1}^N p(\eta_k, \ell_k, t_k \mid x_k; \theta) \prod_{k=1}^N p(s_k \mid \eta_k, \ell_k, t_k, x_k; \theta), \quad (3)$$

corresponding respectively to event timing/type/level and to order size modeling. The MDQR implementation relies on two distinct neural networks. The first network models the multidimensional event intensities and captures the joint dynamics of event timing, category, and price level. The second network models order sizes. Since empirical order size distributions are highly skewed and strongly state-dependent, order sizes are discretized into a finite set of classes. The size model is therefore formulated as a multi-class classification problem, where a neural network predicts the conditional probability of each size class given  $(\eta_k, \ell_k, x_k)$ .

The MDQR model is evaluated on its ability to reproduce key stylized facts of limit order books, including event dynamics, transition structures, and distributional properties. It improves over QR and DQR by capturing cross-level interactions and temporal dependencies, while also achieving strong predictive performance.

Table 2: Neural Architectures of the MDQR Model

| Component         | Intensity Network         | Order Size Network      |
|-------------------|---------------------------|-------------------------|
| Input             | Global state $x_k$        | $(\eta_k, \ell_k, x_k)$ |
| Hidden Layers     | [256, 64]                 | [256, 64]               |
| Hidden Activation | Tanh                      | Tanh                    |
| Output Dimension  | 30 (event $\times$ level) | 200 (size classes)      |
| Output Activation | ReLU                      | Softmax                 |
| Loss Function     | NLL                       | Cross-entropy           |
| Optimizer         | Adam                      | Adam                    |
| Learning Rate     | $10^{-5}$ to $10^{-3}$    | $10^{-5}$ to $10^{-3}$  |

## 2 Numerical experiments

### 2.1 Data and preprocessing

Our empirical study uses LOBSTER data for Intel (INTC), selected as a large-tick stock (spread = 1 tick 75% of the time) whose microstructure approximates Euro-Bund Futures, whose data is unavailable (Appendix A). The dataset consists of synchronized message (events) and order book (queues) files. We map the LOBSTER representation onto the QR framework as described in Huang et al. (2015)[2], which uses a fixed grid centered at  $p_{\text{ref}}$ :

$$P_{\pm i} = p_{\text{ref}} \pm \left(i - \frac{1}{2}\right) \delta, \quad i = 1, \dots, K,$$

allowing queues  $Q_{\pm i}$  to be zero. Each snapshot is re-gridded with  $K = 5$  levels per side and a consistent reference price. Queue sizes are normalized by the average event size per queue ( $\text{AES}_i$ ), and each event is paired with the preceding state for estimation.

### 2.2 Queue-Reactive (QR) Model

For each level  $\ell \in \{-K, \dots, -1, 1, \dots, K\}$ , we construct sequences  $(\eta_k, q_k, \Delta t_k)$ , where  $\eta_k \in \{L, C, M\}$  denotes the event type,  $q_k$  the queue size at level  $\ell$  immediately before the event, and  $\Delta t_k$  the inter-event time. The QR intensities are estimated by maximum likelihood: for each queue size  $q$ , we compute the exposure time  $T(q) = \sum_{k: q_k = q} \Delta t_k$  and event counts  $N^\eta(q) = \#\{k : q_k = q, \eta_k = \eta\}$ , yielding  $\hat{\lambda}^\eta(q) = N^\eta(q)/T(q)$  and  $\hat{\Lambda}(q) = \sum_\eta \hat{\lambda}^\eta(q)$ . This estimation is performed independently for each level.

Figure 1 shows the estimated intensities at the best bid and ask. As in the article, the QR model captures the dependence of order flow on queue size, with higher activity for small queues and lower, more stable rates for larger ones. However, intensities become noisy for large  $q$ , especially at deeper levels, reflecting data sparsity and limited exposure.

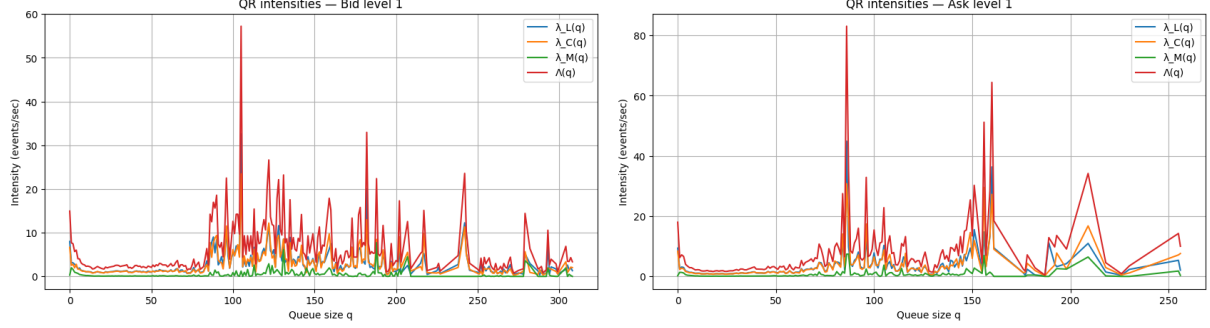


Figure 1: Estimated QR intensities at the best bid and ask.

Figure 2 compares the empirical transition matrix with the one implied by the QR model. While the data exhibits strong dependence on the previous event type (excitation effects), the QR model produces identical rows, indicating no sequential dependence. This highlights a key limitation of the QR framework: it captures local state dependence but fails to reproduce temporal dependencies in order flow.

### 2.3 Deep Queue-Reactive (DQR) Model

The Deep Queue-Reactive (DQR) model extends QR by replacing  $\lambda^\eta(q_k)$  with neural intensities  $\lambda_\theta^\eta(x_k)$  depending on a richer state  $x_k$ . In our implementation,  $x_k$  includes  $q_k$ ,  $h_k$ , and  $\eta_{k-1}$ , capturing intraday seasonality and short-term dependence. We compare four specifications:  $x_k = [q_k]$  (Vanilla),  $[q_k, h_k]$  (Hour),  $[q_k, \eta_{k-1}]$  (Last-event), and  $[q_k, h_k, \eta_{k-1}]$  (Hour + Last-event). Figure 2 focuses on excitation. It compares the empirical transition matrix, the QR matrix, and the DQR matrix with  $x_k = [q_k, h_k, \eta_{k-1}]$ . In the data, transition probabilities depend strongly on the previous event type. By contrast with QR, DQR recovers a transition structure much closer to the empirical one. This shows that including  $\eta_{k-1}$  allows the model to capture short-term event dependence that is absent from QR.

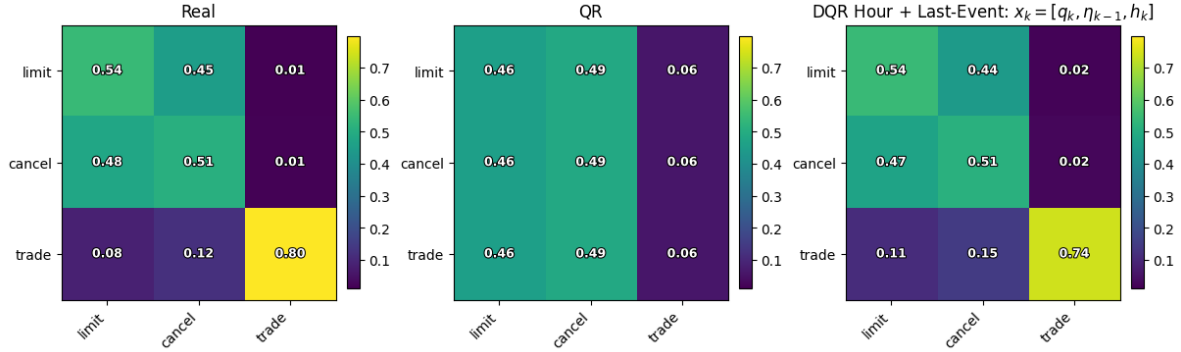


Figure 2: Transition matrices for empirical data, QR, and DQR with  $x_k = [q_k, \eta_{k-1}, h_k]$ .

Figure 3 focuses on intraday seasonality. It compares empirical hourly trade intensities with QR and DQR. As expected, QR remains essentially flat because it does not include time-of-day information. DQR with hour features follows the observed hourly variation much more closely, showing that  $h_k$  helps the model adapt to changing market activity across the trading day.

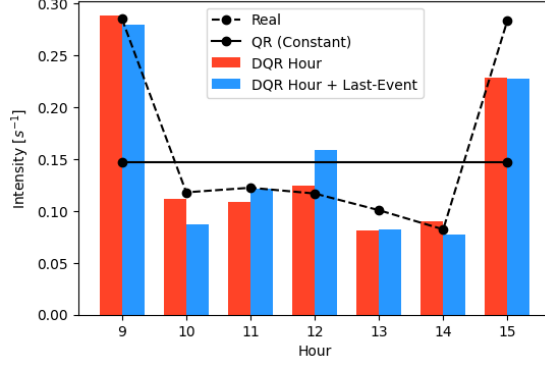


Figure 3: Hourly trade intensity: empirical data, QR, and DQR models with hour-based features.

Finally, Figure 4 summarizes the quantitative results. Enriching the state space improves the mean log-likelihood and the balanced accuracy of next-event prediction, while reducing the relative error on the time to the next event. The best overall specification is  $x_k = [q_k, h_k, \eta_{k-1}]$ , which confirms that hour-of-day and previous-event information provide complementary information beyond queue size alone.

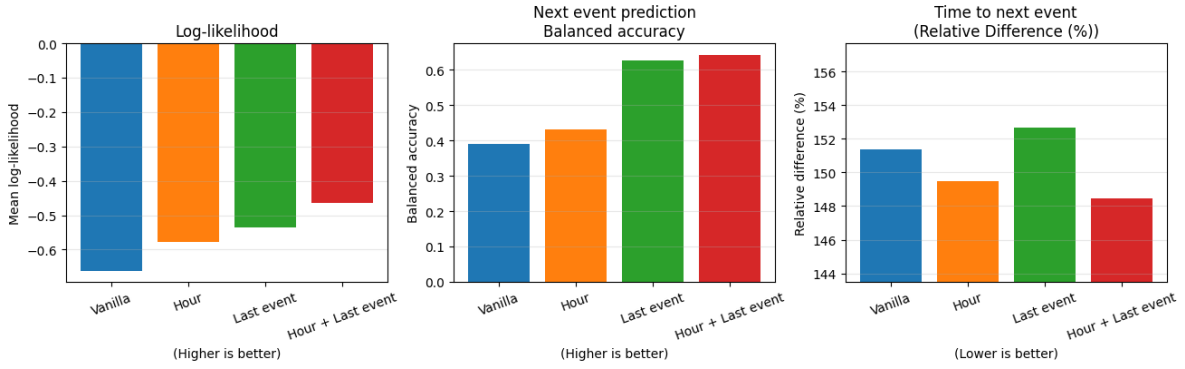


Figure 4: Comparison of DQR specifications across log-likelihood, next-event balanced accuracy, and relative error on time-to-next-event prediction.

## 2.4 Multidimensional Deep Queue-Reactive (MDQR) Model

We implement the MDQR model as described in Section 1.3, with two adaptations. First, the state vector combines both per-level and global last-event embeddings, providing richer temporal context across price levels. Second, we follow the same approach as the original paper to define size classes, covering 99.9% of normalized order sizes. Due to the more concentrated distribution of INTC data, this results in 22 classes instead of 200 (Appendix B).

**Training :** Both networks converge stably with no significant train/validation gap (Figure 5), indicating limited overfitting.

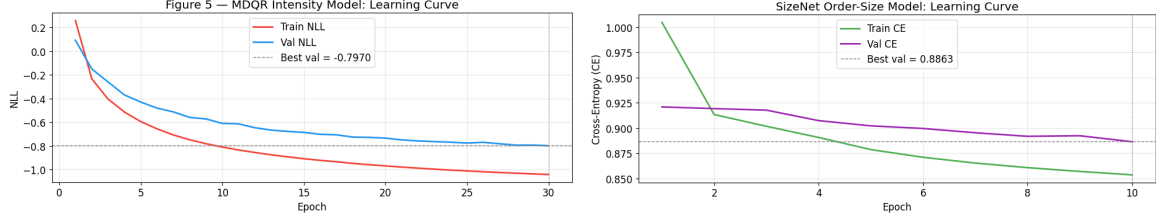


Figure 5: Learning curves for the MDQR intensity network (left) and SizeNet (right).

**Results :** We evaluate the model along four dimensions using a full-day simulation.

*Event distribution.* The model closely matches empirical probabilities across the 30 event classes (Figure 6). Best-level events dominate and are well reproduced, while activity decays correctly with depth on both sides, showing that the model captures the spatial structure of order flow.

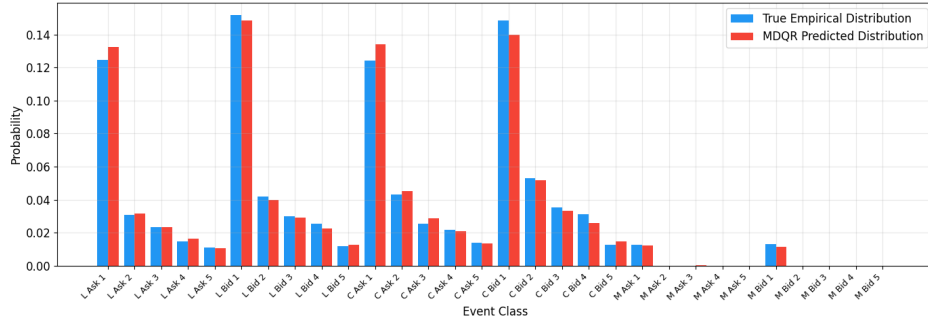


Figure 6: Empirical vs. predicted marginal distributions across the 30 event classes.

*Sequential dependence.* Figure 7 highlights two regimes. Trade events are strongly self-exciting and well reproduced (e.g. 0.63 vs. 0.70 for Ask Trade  $\rightarrow$  Ask Trade). In contrast, same-side persistence of passive events is underestimated (e.g. 0.23 vs. 0.33), suggesting that a single last-event feature is insufficient to capture longer-term clustering.

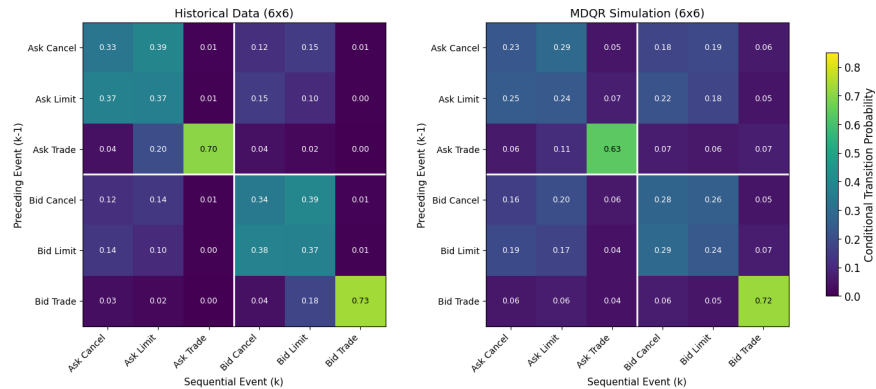


Figure 7: 6  $\times$  6 event transition matrices: historical (left) vs. MDQR simulation (right).

*Queue size distribution.* The simulated distribution at Ask Level 1 closely matches the empirical one (Figure 8), with good agreement up to the 90th percentile. The heaviest tails are underestimated, reflecting the

difficulty of reproducing rare high-liquidity states.

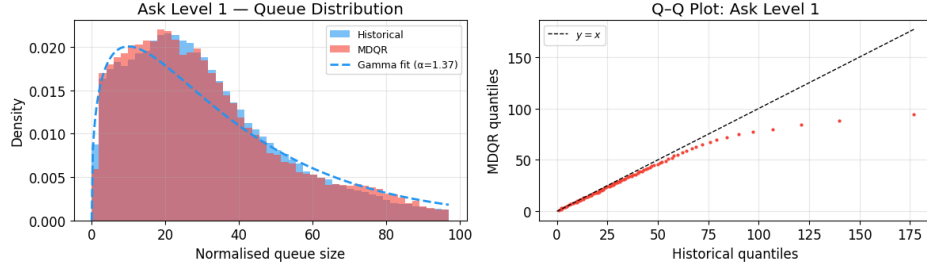


Figure 8: Queue size distribution at Ask Level 1: density histogram (left) and Q-Q plot (right). The simulation closely tracks the empirical distribution up to the 90th percentile.

*Market impact.* The simulated impact follows a concave, near square-root scaling during execution ( $\hat{\alpha} = 0.45$ ), consistent with empirical evidence (Figure 9). However, post-execution price drift is excessive due to the queue-replenishment rule, which assigns a fixed median size after price shifts. Alternative sampling schemes were tested (e.g. empirical sampling from Gamma distribution, Huang et al.(2015)[2]) but degraded other metrics, so the median rule is retained.

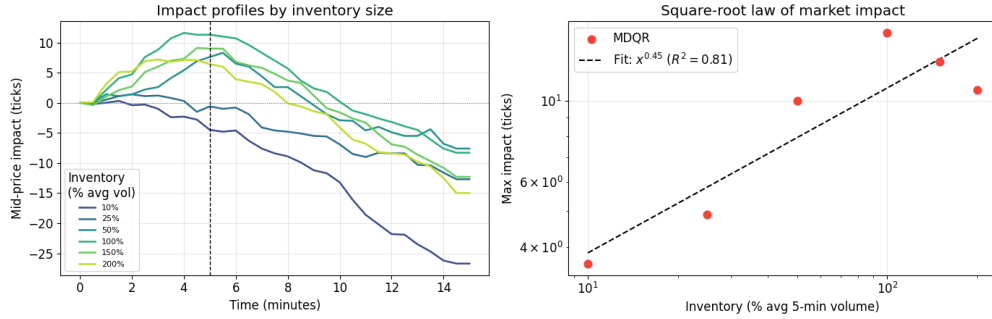


Figure 9: Market impact profiles and scaling.

Overall, the MDQR framework successfully reproduces the dominant microstructural properties of order flow. The main residual limitation on the modeling side insufficient memory for passive event clustering is discussed further in Section 4.

### 3 Can MDQR learn rough volatility patterns ?

After reproducing several stylized facts from Bodor and Carlier (2025) [1], we wanted to extend the analysis by investigating whether the MDQR simulator can also reproduce other important stylized facts of LOB, in particular key properties of intraday volatility. Although volatility is not directly included in the training objective, a realistic simulator should generate price trajectories with volatility clustering, intraday seasonality, and, ideally, the roughness signature documented in Gatheral, Jaisson and Rosembaum (2014) [3].

### 3.1 Realized volatility estimation

Following Rosenbaum's framework [4], we estimate the efficient price from observed prices:

$$X_{\tau_i} = P_{t_i} - \alpha \left( \frac{1}{2} - \eta \right) \text{sign} (P_{t_i} - P_{t_{i-1}}), \quad \hat{X}_{\tau_i} = P_{t_i} - \alpha \left( \frac{1}{2} - \hat{\eta} \right) \text{sign} (P_{t_i} - P_{t_{i-1}}).$$

Here,  $P_{t_i}$  is the observed price and  $\hat{X}_{\tau_i}$  the estimate of the efficient price  $X_{\tau_i}$ . The parameter  $\hat{\eta}$  captures microstructure frictions and is estimated from price continuations vs. alternations. We compute  $\hat{\eta}_t$  over 5-minute rolling windows as described in Appendix D, and obtain  $\hat{X}_{\tau_i}$  accordingly. The realized volatility is then defined as

$$\widehat{RV}_t := \sum_{i=1}^{N_{\alpha,t}} \left( \log \hat{X}_{\tau_i} - \log \hat{X}_{\tau_{i-1}} \right)^2. \quad (4)$$

Figure 10 shows that MDQR reproduces intraday seasonality (higher volatility at open and close) and matches the overall magnitude, supporting its use for execution analysis (Table 5, Appendix E).

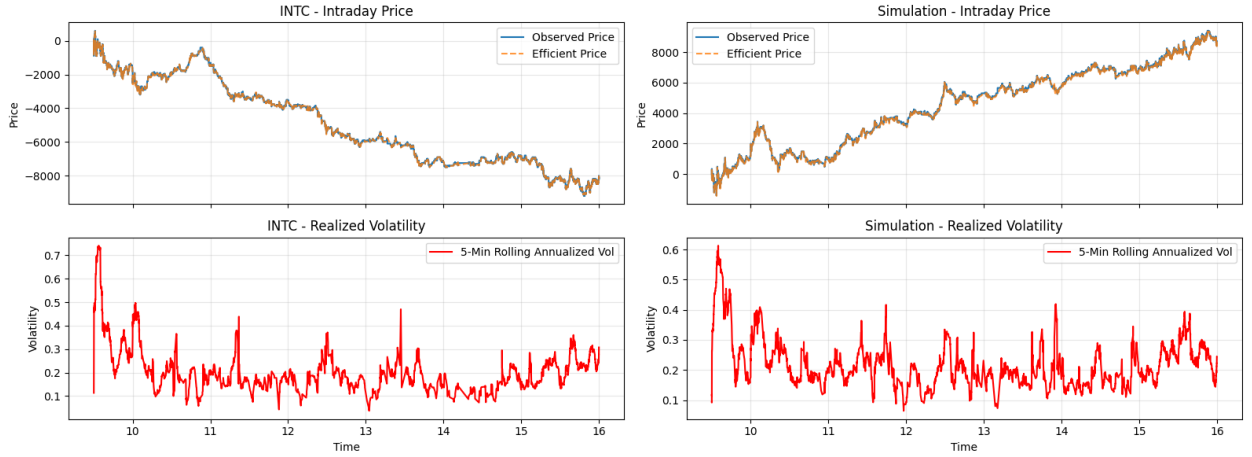


Figure 10: Real (**left**) and simulated (**right**) intraday price dynamics  $P_{t_i}$ , estimated efficient price  $\hat{X}_{\tau_i}$  and 5-min rolling window realized volatility  $\widehat{RV}_t$ .

### 3.2 Hurst exponent estimation

Recent research in volatility modeling suggest that realized volatility, accross multiple time scales (daily and intraday), exhibits a *rough signature*, i.e. volatility behave as a rough stochastic process (Gatheral, Jaisson and Rosenbaum (2014) [3]). The method assumes access to discrete observations of realized volatility on a grid of mesh  $\Delta$  on  $[0, T]$ :  $\sigma_0, \sigma_\Delta, \dots, \sigma_{k\Delta}, \dots, k \in \{0, \lfloor T/\Delta \rfloor\}$ . If  $N = \lfloor T/\Delta \rfloor$ , then for  $q \geq 0$ , [3] defines the function:

$$m(q, \Delta) = \frac{1}{N} \sum_{k=1}^N \left| \log(\sigma_{k\Delta}) - \log(\sigma_{(k-1)\Delta}) \right|^q. \quad (5)$$

Using the trivial partition  $\pi^* = (\pi_N^*)_{N \geq 1}$  of  $[0, T]$  with  $T = 1$  trading day:  $\pi_N^* = (t_i^N)_{i=1}^N$ ,  $t_i^N = \frac{iT}{N}$ ,  $i = 1, \dots, N$ , from one of our previous work [5], we adapt (5) to our framework, similar to the the

work of Cont and Das (2024) [6]:

$$m(q, \Delta) := \frac{1}{N} \left[ \log \widehat{RV} \right]_{\pi_N^*}^{(q)} = \frac{1}{N} \sum_{k=1}^N \left| \log(\widehat{RV}_{t+\Delta}) - \log(\widehat{RV}_t) \right|^q \approx C_q \Delta^{\xi_q}. \quad (6)$$

Gatheral, Jaisson and Rosembaum (2014) [3] show that if  $\widehat{H}_R$  denotes the estimator of the Hurst exponent of realized volatility process, then the scaling exponents  $\xi_q$  behave linearly in  $q$  as follows:

$$\xi_q \approx q \widehat{H}_R. \quad (7)$$

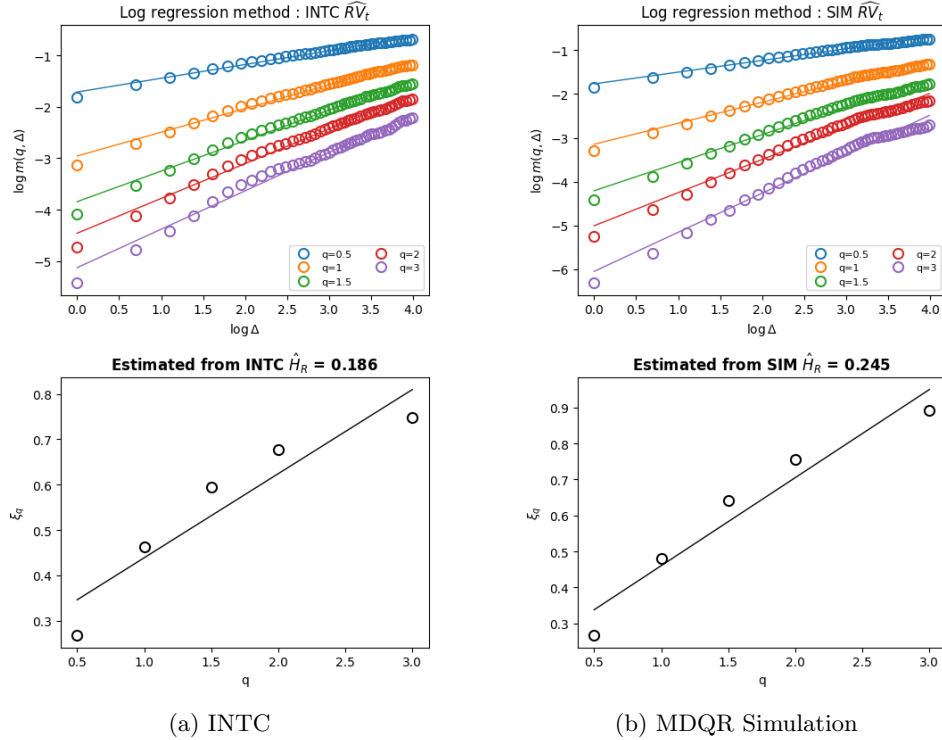


Figure 11:  $\log m(q, \Delta)$  as a function of  $\log \Delta$  (**top**) and  $\widehat{H}_R$  as regression of  $\xi_q$  over  $q$  (**bottom**).

### 3.3 Observations

Empirically, Gatheral, Jaisson and Rosembaum (2014)[3] estimated the Hurst exponent for several equity indices, in their appendix B.2, using the Oxford-Man Institute's Realised Library<sup>1</sup>. They report values such as  $\widehat{H}_{SPX} = 0.158$ ,  $\widehat{H}_{FTSE} = 0.156$ ,  $\widehat{H}_{STOXX50E} = 0.132$ . In particular,  $\widehat{H}_{IXIC} = 0.156$  for the NASDAQ Composite Index, on which INTX is traded. Figure 11 compares the Hurst exponent estimated from INTX data and from the MDQR simulation using the log-regression method. The empirical estimate  $\widehat{H}_{INTX} = 0.186$  is consistent with the literature. The MDQR simulation yields a higher value,  $\widehat{H}_{SIM} = 0.245$ , indicating smoother and more persistent volatility.

These results show that MDQR is able to capture the roughness of volatility from order book data alone, while also reproducing realistic intraday dynamics such as seasonality and liquidity fluctuations. However, the

<sup>1</sup>See <https://oxford-man.ox.ac.uk/research/realised-library/>

higher Hurst exponent suggests a tendency to overestimate persistence, which may lead to an underestimation of short-term fluctuations and execution risk, and could be link to the limitations discussed in *Market impact* of Section 2.4. These observations motivate a broader discussion of the strengths and limitations of the MDQR model.

## 4 Critical Discussion

The MDQR framework extends previous approaches by jointly modeling event intensities across the full order book, capturing dynamics that per-queue models can not. However, several design choices still present limitations.

**Limited temporal memory.** The model encodes only the most recent event, which is sufficient to capture short-term excitation effects but remains a limited representation of market memory. Order flow exhibits longer-range dependencies that cannot be captured by a single lag. More expressive sequential architectures, such as RNNs or Transformers, could allow the model to incorporate richer temporal information.

**Spatial structure.** The LOB has a natural spatial organization that is not explicitly exploited by the MLP architecture used in MDQR. A natural extension is to use convolutional networks to capture local interactions across price levels. We implemented a CNN variant (Appendix C), but it performed worse than the baseline MLP, especially on transition dynamics (e.g. Ask Trade  $\rightarrow$  Ask Trade drops from 0.63 to 0.39). This suggests that, in this setting, the spatial structure is either too limited ( $2K = 10$ ) or already implicitly captured by dense layers. While promising, this direction would require further investigation.

**Difficulty in modeling rare events.** Likelihood-based training favors frequent events, leading to an under-representation of rare but impactful ones. This limits the model’s ability to capture tail behavior and suggests the need for dedicated mechanisms to better handle low-probability events.

**Single-regime intensity modeling.** The model relies on a single set of intensities, while market activity alternates between different regimes. This limits its flexibility and suggests that regime-switching or mixture models could better capture such heterogeneity.

**Volatility dynamics** The MDQR simulator reproduces key volatility features such as intraday seasonality and roughness, making it relevant for modeling realistic execution risk . However, the slightly higher Hurst exponent indicates overly smooth volatility, suggesting that incorporating richer temporal dependencies could improve the capture of short-term fluctuations.

## Conclusion

We reproduced and extended the MDQR framework, showing that deep learning significantly improves the modeling of limit order book dynamics compared to classical queue-based approaches. The model captures key stylized facts such as excitation effects, intraday seasonality, and realistic volatility patterns.

However, limitations remain, notably limited temporal memory and difficulty in modeling rare events. Future work could address these issues using sequence models or regime-switching approaches. Overall, MDQR provides a powerful simulator capable of reproducing complex market dynamics.

## References

- [1] Hamza Bodor and Laurent Carlier. Deep learning meets queue-reactive: A framework for realistic limit order book simulation. *arXiv preprint arXiv:2501.08822*, 2025.
- [2] Weibing Huang, Charles-Albert Lehalle, and Mathieu Rosenbaum. Simulating and analyzing order book data: The queue-reactive model. *Journal of the American Statistical Association*, 110(509):107–122, 2015.
- [3] Jim Gatheral, Thibault Jaisson, and Mathieu Rosenbaum. Volatility is rough. *arXiv preprint arXiv:1410.3394*, 2014.
- [4] Mathieu Rosenbaum. Modelling ultra high frequency financial data: Model with uncertainty zones and associated statistical procedures for volatility, correlation and lead-lag, 2025. Lecture notes.
- [5] Thibault Marty, Alistair Lucet, Kavya Shah, and Orkhan Ashrafov. Rough volatility: fact or artefact? cont et das (2024) - reproduction and statistical assessment of published results., 2026. MSc Mathematics and Finance, Statistical Methods for Finance.
- [6] Rama Cont and Purba Das. Rough volatility: Fact or artefact? *Sankhya B: The Indian Journal of Statistics*, 86(1):191–223, 2024.

# Appendix

## A INTC : Summary Statistics

INTC is a large-tick stock on NASDAQ: the bid-ask spread equals one tick 75% of the time and averages 1.25 ticks, making its microstructure comparable to the Euro-Bund Futures studied in the original paper. Table 3 reports key statistics over the sample period.

Table 3: INTC summary statistics (LOBSTER, Sep–Oct 2013, 23 trading days).

| Statistic                          | Value                   |
|------------------------------------|-------------------------|
| Tick size                          | \$0.01                  |
| Average mid-price                  | \$23.39                 |
| Average spread                     | 1.25 ticks              |
| % of time spread = 1 tick          | 75.2%                   |
| Total events                       | 10,127,006              |
| Avg per day                        | 440,305                 |
| Limit orders                       | 48.9%                   |
| Cancellations                      | 47.2%                   |
| Market orders                      | 3.9%                    |
| Avg inter-event time (mean/median) | 53.3 / 0.03 ms          |
| Avg AES-normalised queue (Level 1) | 30.3 (bid) / 32.1 (ask) |

## B Normalised Order Size Distribution

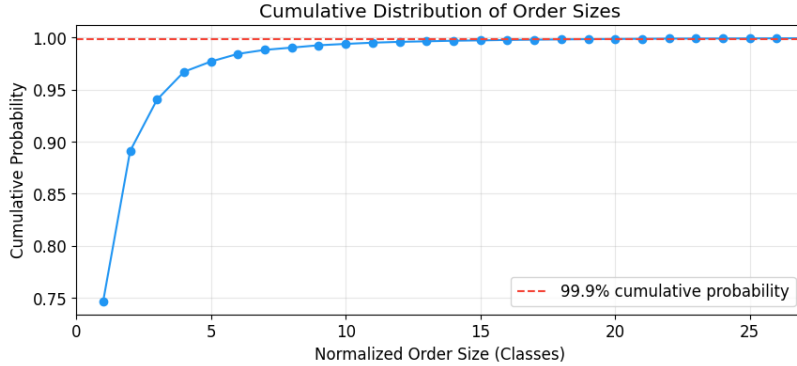


Figure 12: Cumulative distribution of AES-normalised order sizes for INTC. The dashed line marks the 99.9% threshold (22 classes).

## C CNN Variant of MDQR

Motivated by the spatial structure of the LOB, we implemented convolutional variants of both MDQRNet and SizeNet. In each case, the  $2K = 10$  AES-normalised queue sizes bid levels reversed and concatenated with ask levels to form a symmetric price-ordered sequence are processed by two `Conv1d` layers before being passed to the standard MLP head. The rest of the architecture (embeddings, other continuous features, output layer) is unchanged.

Table 4: CNN variant: modifications to MDQRNet and SizeNet.

| Component     | MDQRNet (CNN)                                                              | SizeNet (CNN)                  |
|---------------|----------------------------------------------------------------------------|--------------------------------|
| Spatial input | $[q_{-K}^{\text{rev}}, \dots, q_{-1}, q_1, \dots, q_K]$ , length $2K = 10$ |                                |
| Conv block 1  | Conv1d( $1 \rightarrow 8$ , kernel=3, pad=1) + Tanh                        |                                |
| Conv block 2  | Conv1d( $8 \rightarrow 16$ , kernel=3, pad=1) + Tanh                       |                                |
| Conv output   | $16 \times 10 = 160$ features                                              |                                |
| MLP head      | $[256, 64]$ + Tanh, Softplus                                               | $[256, 64]$ + Tanh, 22 classes |

The results are mixed. On the static side, the CNN reproduces the queue size distribution at Ask Level 1 almost as well as the MLP (Figure 13), confirming that the convolutional filters do extract meaningful local structure from the queue sequence. On the dynamic side, however, the transition matrix deteriorates significantly (Figure 14): Ask Trade  $\rightarrow$  Ask Trade drops from 0.63 to 0.39 and Bid Trade  $\rightarrow$  Bid Trade from 0.72 to 0.35, while cross-side transitions are over-estimated.

We conjecture that this failure is specific to the intensity modeling task. Trade self-excitation is driven by the depletion of the best-level queue a sharp, local signal that a dense layer can detect directly via a single large weight on  $q_1$ , whereas a convolutional filter averages over a neighborhood, potentially smoothing out precisely the signal that matters. More generally, the sequence length  $2K = 10$  is arguably too short for convolutions to provide a meaningful inductive bias over a well-tuned MLP. All results in the main text use the original MLP architecture.

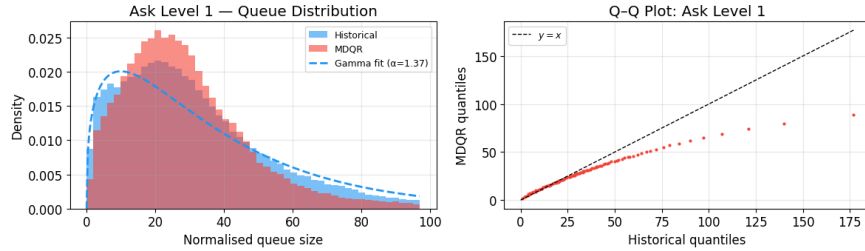


Figure 13: Queue size distribution at Ask Level 1 for the CNN variant. Results are comparable to the MLP baseline (Figure 8), confirming that the CNN failure is specific to the intensity task.

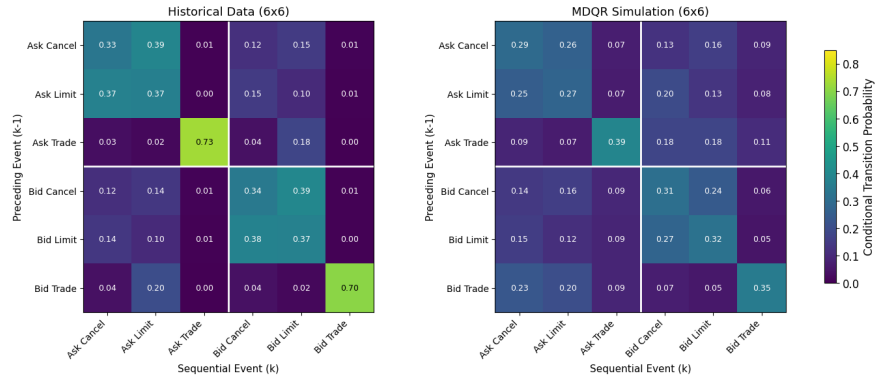


Figure 14:  $6 \times 6$  transition matrices for the CNN variant: historical (left) vs. simulation (right). Trade self-excitation is significantly underestimated relative to the MLP baseline (Figure 7).

## D $\hat{\eta}_t$ estimation

If we follow Rosenbaum's framework [4], we estimate an efficient price from the observed price process:

$$X_{\tau_i} = P_{t_i} - \alpha \left( \frac{1}{2} - \eta \right) \text{sign} (P_{t_i} - P_{t_{i-1}}), \quad \hat{X}_{\tau_i} = P_{t_i} - \alpha \left( \frac{1}{2} - \hat{\eta} \right) \text{sign} (P_{t_i} - P_{t_{i-1}})$$

Here,  $P_{t_i}$  denotes the observed price,  $X_{\tau_i}$  the efficient price, and  $\hat{X}_{\tau_i}$  its estimate (see Figure 15, Appendix). The parameter  $\hat{\eta}$  captures microstructure frictions, with smaller values corresponding to stronger mean reversion.

The estimation of  $\hat{\eta}$  is based on the cardinality of *continuations* (a price variation in the same direction to the one of the preceding variation) and *alternations* (a price variation in the opposite direction to the one of the preceding variation). Denote  $N_{\alpha,t,k}^{(c)}$  and  $N_{\alpha,t,k}^{(a)}$  respectively the number of continuations and alternations of size  $\alpha k$  in an interval  $[0, t]$  of time. Then let  $N_{\alpha,t} = \text{card}\{t_i; t_i \leq t\}$ . For  $k = 1, \dots, m$ ,

$$N_{\alpha,t,k}^{(c)} = \sum_{i=1}^{N_{\alpha,t}} \mathbb{1}\{|X_{\tau_i} - X_{\tau_{i-1}}| = \alpha k\},$$

$$N_{\alpha,t,k}^{(a)} = \sum_{i=1}^{N_{\alpha,t}} \mathbb{1}\{|X_{\tau_i} - X_{\tau_{i-1}}| = \alpha(k-1+2n)\}.$$

The friction coefficient estimator is computed over 5-minute rolling windows as:

$$\hat{\eta}t = \sum_{k=1}^m \lambda_{\alpha,t,k} u_{\alpha,t,k},$$

$$\text{with } \lambda_{\alpha,t,k} = \frac{N_{\alpha,t,k}^{(a)} + N_{\alpha,t,k}^{(c)}}{\sum_{j=1}^m [N_{\alpha,t,j}^{(a)} + N_{\alpha,t,j}^{(c)}]}, \quad \text{and } u_{\alpha,t,k} = \frac{1}{2} \left( k \left( \frac{N_{\alpha,t,k}^{(c)}}{N_{\alpha,t,k}^{(a)}} - 1 \right) + 1 \right).$$

The parameter  $\hat{\eta}$  is estimated using Rosenbaum methodology based on the relative frequency of price continuations and alternations. This provides a microstructure-adjusted estimate of the efficient price. We compute  $\hat{\eta}_t$  on a 5-minute rolling window and use it to estimate the efficient price  $\hat{X}_{\tau_i}$ . An estimate of the microstructure realized volatility over the rolling window is given for all  $t$  in a trading day by:

$$\widehat{RV}t := \sum_{i=1}^{N_{\alpha,t}} \left( \log(\hat{X}_{\tau_i}) - \log(\hat{X}_{\tau_{i-1}}) \right)^2. \quad (8)$$

Figure 15 shows the difference between the true price (Last Traded Price) and the estimation of the efficient price trajectories over a short window.

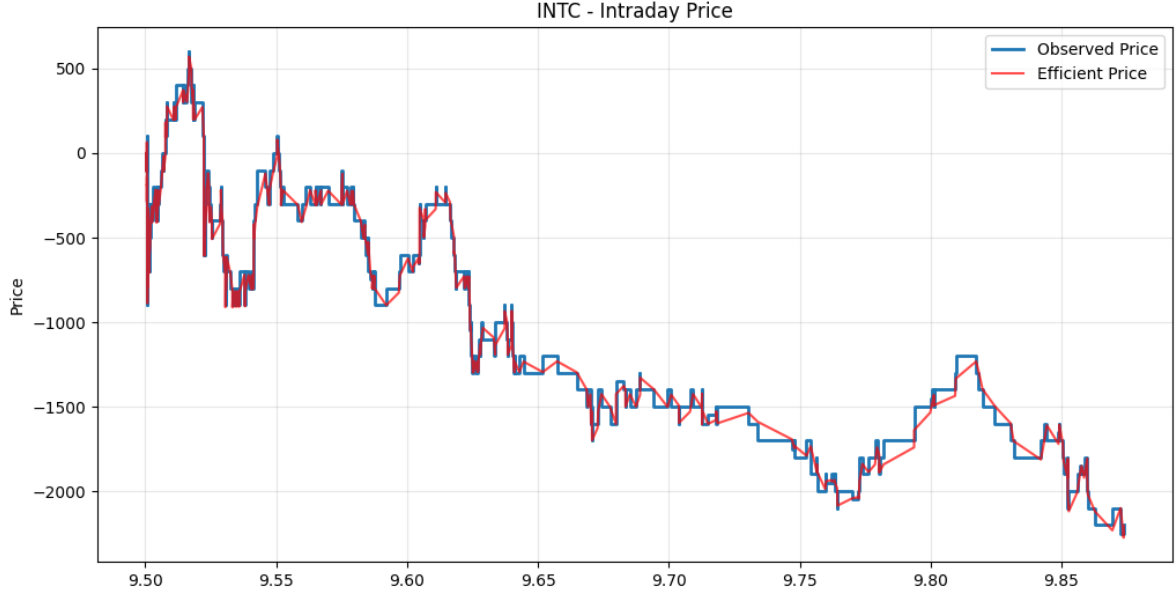


Figure 15: Estimated efficient price  $\hat{X}_t(\hat{\eta}_t, t)$  (smooth) vs the real last traded price  $P_t$  (steps).

## E Intraday volatility clustering

| Period         | Realized vol from INTC | Simulated vol from MDQR | Relative error (%) |
|----------------|------------------------|-------------------------|--------------------|
| Market opening | 0.3929                 | 0.3422                  | -12.91             |
| Middle of day  | 0.1735                 | 0.1944                  | +12.02             |
| Market closing | 0.2344                 | 0.2411                  | +2.86              |

Table 5: Comparison of real and simulated volatility with relative error. Market open (9h30 to 10h30), Middle of day (10h30 to 15h30), Market close (15h30 to 16h30).